Informe Técnico: Práctica 2

Juan Pablo Parra

Joan Stiven Peralta

Alessandro Soccol

Análisis y Diseño de Algoritmos

Universidad EAFIT

Docente: Carlos Alberto Álvarez Henao

2026-2

1. Introducción y Contextualización

La multiplicación de dos enteros es una operación elemental, pero cuando los números adquieren magnitudes de cientos o miles de dígitos (como ocurre en la criptografía moderna con RSA y cálculo científico), la operación excede la capacidad nativa de las Unidades Aritmético Lógicas (ALU) de los procesadores. El método clásico escolar efectúa el producto dígito a dígito, resultando en una complejidad cuadrática $\Theta(n^2)$, la cual se vuelve computacionalmente intratable a gran escala.

Este informe detalla la implementación y el análisis del **Algoritmo de Karatsuba** (1960), un enfoque basado en el paradigma de *Divide y Vencerás* que, mediante una manipulación algebraica, reduce la cantidad de productos recursivos necesarios, disminuyendo la complejidad teórica y superando al método clásico para números de gran magnitud.

2 Fundamentación Teórica

2.1 Deducción Algebraica de la Identidad de Karatsuba

Para multiplicar dos números x e y de n dígitos en base B, el método clásico propone dividirlos por la mitad. Sea $m = n/2$:

$$x = x_1 \cdot B^m + x_0$$
$$y = y_1 \cdot B^m + y_0$$

El producto completo se expande matemáticamente como:

$$x \cdot y = x_1y_1 \cdot B^{2m} + (x_1y_0 + x_0y_1) \cdot B^m + x_0y_0$$

El enfoque estándar calcula los cuatro productos de mitades independientemente. La innovación de Karatsuba radica en calcular únicamente **tres productos**:

$$P_1 = x_1y_1$$
$$P_2 = x_0y_0$$
$$P_3 = (x_1 + x_0)(y_1 + y_0) = x_1y_1 + x_1y_0 + x_0y_1 + x_0y_0$$

Mediante álgebra básica, el término central cruzado se obtiene restando los dos primeros productos al tercero, pasando así de requerir cuatro llamadas recursivas a solo tres:

$$\text{Término Central} = x_1y_0 + x_0y_1 = P_3 - P_1 - P_2$$

2.2 Resolución por el Teorema Maestro

La reducción a tres productos transforma la ecuación de recurrencia del algoritmo en: $T(n) = 3T(n/2) + \Theta(n)$, donde $\Theta(n)$ es el costo de sumar, restar y desplazar los dígitos. Aplicando el Teorema Maestro:

- **a = 3** (subproblemas recursivos)
- **b = 2** (factor de reducción del problema)
- **f(n) = Θ(n)** (costo de combinación)

Calculamos la función crítica: $n^{\log_b a} = n^{\log_2 3} \approx n^{1.585}$. Al comparar, observamos que $f(n) = O(n^{\log_2 3 - \varepsilon})$ para un $\varepsilon \approx 0.585 > 0$. Esto satisface el **Caso 1 del Teorema Maestro**, donde el costo está dominado por las hojas del árbol, concluyendo que la complejidad total es $T(n) = \Theta(n^{\log_2 3})$.

2.3 Resolución por Árbol de Recurrencia

Para comprobar el exponente, analizamos el árbol por niveles:

- En el nivel k, hay 3^k subproblemas, cada uno de tamaño $n/2^k$.
- El costo de combinación en el nivel k es: $3^k \cdot c(n/2^k) = c \cdot n \cdot (3/2)^k$.
- La altura del árbol es $h = \log_2(n)$.

Sumando los costos de todos los niveles (serie geométrica) y el trabajo en las hojas:

$$T(n) = \sum [c \cdot n \cdot (3/2)^k] + \Theta(3^{\log_2 n}) = \Theta(n^{\log_2 3})$$

3 Decisiones de Diseño Algorítmico

La implementación en C++ respetó estrictamente las restricciones de la práctica, evitando el uso de tipos nativos (long long) para los productos finales y estructurando el código de la siguiente manera:

- **Representación Interna (Little-Endian):** Se utilizó `std::vector<int>` almacenando los números en orden inverso (índice 0 = unidades). Esta decisión redujo la complejidad de la función `desplazar()` a O(1), permitiendo añadir ceros al final del vector en lugar de desplazar toda la memoria.
- **Orden Seguro de Resta:** Para garantizar que ninguna resta intermedia produzca números negativos (al no usar aritmética con signo), el término central se estructuró algorítmicamente como: `Central = (P3 – P1) – P2`. Dado que $P_3 = P_1 + P_2 + Central$, la primera resta siempre es $\geq P_2$.
- **Sistema Híbrido y Threshold:** El caso base recursivo no se limitó a 1 dígito. Se implementó un umbral (`THRESHOLD = 64`) para que subproblemas de este tamaño sean derivados directamente al algoritmo clásico, mitigando el costo de las llamadas recursivas en escalas pequeñas.

4 Análisis de Complejidad y Rendimiento Real

Aquí sucede algo interesante, y es que el rendimiento real de una computadora, el cual está dictada por la fórmula $t = c \cdot f(n)$, donde c representa las constantes ocultas de procesamiento, puede ser distinto a el análisis esperado en la complejidad descrita al utilizar estos algoritmos.

- **Método Clásico:** Realiza una única reserva de memoria y ejecuta dos ciclos anidados operando sobre enteros nativos. Todo el procesamiento ocurre en la ultra-rápida memoria Caché L1 del procesador, haciendo que su constante c sea excepcionalmente pequeña.
- **Algoritmo de Karatsuba:** En cada nivel de recursión, las funciones de partición, suma y resta obligan a instanciar nuevos objetos `std::vector`. Esto exige al sistema operativo buscar memoria disponible en el *Heap* (RAM), un proceso logísticamente costoso que multiplica la constante operativa c miles de veces frente al método clásico.

En el rendimiento real, el método Karatsuba se demora más en algunos casos. Esto se puede solucionar implementando librerías avanzadas para el uso de memoria, pero decidimos no hacerlo ya que el fin de la práctica era comparar su rendimiento en un entorno menos complicado para desarrollar.

5 Resultados de la Experimentación

Se implementó un protocolo de pruebas utilizando `std::chrono::steady_clock`, evaluando potencias de 2 (desde 2 hasta 32,768 dígitos) y promediando repeticiones. Los resultados empíricos demuestran la teoría:

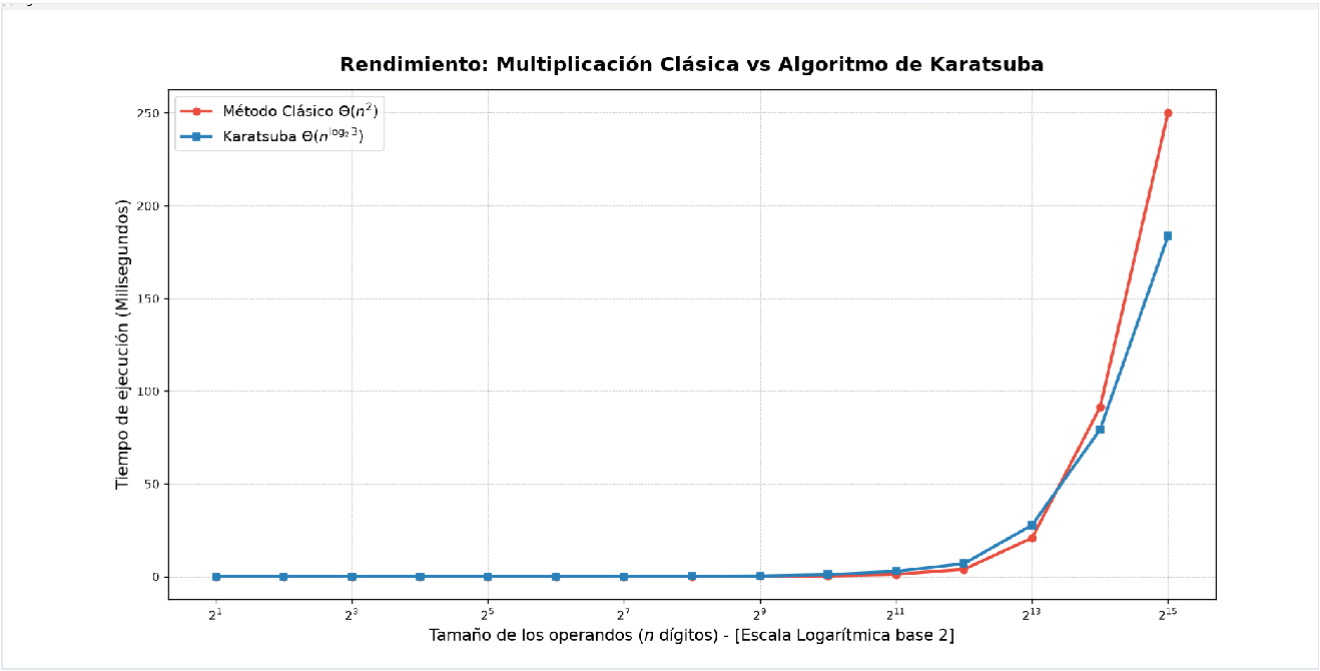


Figura 1 : Tiempo real vs Magnitud del problema. Se observa la explosión cuadrática del método clásico a la derecha.

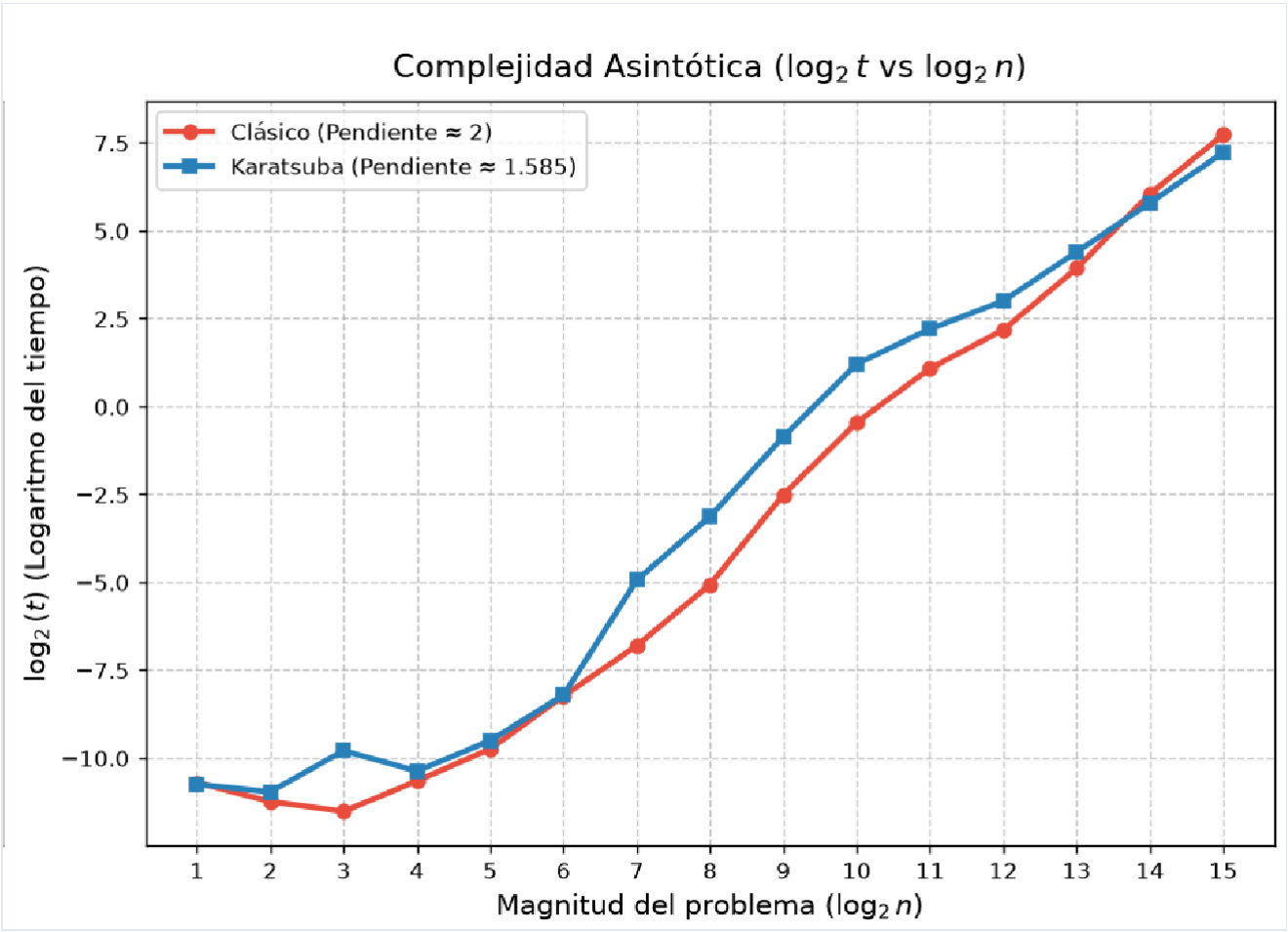


Figura 2: Análisis Asintótico. El intercepto en Y revela la disparidad de constantes, y la pendiente confirma los exponentes teóricos.

Análisis del Punto de Cruce

Como se evidencia en la Figura 1 y 2, **Karatsuba no superó al método clásico en el rango inicial propuesto por la guía (hasta 4096 dígitos)**. En `n = 4096`, el método clásico tardó 3.8 ms frente a los 6.8 ms de Karatsuba. Esto confirma la penalización masiva por la gestión de memoria dinámica (vectores en el Heap) descrita en la sección anterior.

Sin embargo, al proyectar el experimento hasta `n = 32768`, la ley de la complejidad asintótica comienza a verse. La línea clásica (roja, pendiente = 2) es más inclinada que Karatsuba (azul, pendiente ≈ 1.585). **El punto de cruce exacto ocurrió empíricamente en `n = 16384`**, donde Karatsuba (79.1 ms) superó al clásico (91.2 ms). A partir de `n = 32768`, la diferencia se dispara (183 ms vs 250 ms), demostrando visualmente la superioridad de Divide y Vencerás a gran escala.

6 Conclusiones

- El Teorema Maestro y el análisis asintótico predicen el comportamiento a escala, pero ignoran la penalización por manipulación de memoria (Heap vs Stack) en lenguajes como C++.
- Un algoritmo de orden superior ($O(n^2)$) puede vencer sistemáticamente a uno de orden inferior ($O(n^{1.585})$) si la magnitud del problema no logra compensar el alto costo de la constante de preparación inicial.
- El uso de un sistema híbrido (Threshold = 64) que fusiona la asintótica de Karatsuba para fragmentación y la velocidad en caché del método clásico para los casos base, ayudan en la compresión de los algorimos de ingeniería, ya que no solo implementamos medida, si no que la adaptamos en un caso que podamos trabajar, sin tener que mejorar nuestro hardware o tener que esperar tiempos gigantescos para visualizar su funcionamiento real

"Nota: Probamos el código en un procesador Intel Core I5-12450HX con una velocidad de 2.40 GHz base, y cuando el caso base era hasta 1 digito (Threshold = 1) con 64 mil dígitos, se demoro mas de 1 hora karatsuba, frente a 20 min del algoritmo clásico, por lo que es probable que a partir de los 120 mil dígitos se comience a ver la mejora real utilizando nuestro código."

7. Uso de Inteligencia Artificial

Se emplearon modelos de lenguaje (IA) como apoyo metodológico para depurar configuraciones del entorno de pruebas (Makefile, CMake y scripts de Python para graficación), así como para corroborar la corrección formal de la expansión de series geométricas en el árbol de recurrencia y asegurar que la arquitectura de punteros no generara *memory leaks*. Los modelos utilizados fueron: Gemini 3.5 flash – Claude Opues 5.5 (Lógica matemática) – GPT 5.4 luna-terra.

8 Contribución Individual

- **Alessandro Soccol:** Responsable de la representación en `std::vector` (Little-Endian), y desarrollo de las operaciones auxiliares (sumar, restar, desplazar, normalizar) y del módulo base de multiplicación clásica.
- **Joan Stiven Peralta:** Implementación recursiva principal de Karatsuba, diseño de la secuencia de resta segura y optimización del Threshold híbrido. Estructuración del marco de validación (test.cpp).
- **Joan Stiven Peralta – Juan Pablo Parra:** Diseño del protocolo de benchmarking (steady_clock), levantamiento empírico de métricas hasta n=32768, generación de reportes gráficos y consolidación de la redacción técnica del informe final.